Thematic Assessment Report
Phase 2

TA REPORT PHASE 2

Persistent Memory Architecture in Agents Using DBMS

Survey, Requirements, and Database Design

Submitted in partial fulfilment of course requirements
for Database Management Systems

Bachelor of Technology
Computer Science and Engineering
School of Engineering and Technology

Submitted to:
Dr. Poonam Sangwan
Faculty, DBMS
Department of CSE / SOET
Date of submission: _____

Submitted by:
Harshit Khemani (241302081)
Kush Ahuja (241302122)
Kushagra Agrawal (241302109)
Madhav Bassi (241302100)
B.Tech CSE (AI/ML), Section C

https://github.com/HKTITAN/dbms-agent-memory

SGT University
School of Engineering and Technology

Declaration. We declare that this report is our own work for DBMS TA-2.

1. Introduction and objective

Phase 1 proposed a persistent memory for an AI agent, built as a relational database. It covered the problem, the objectives, the scope, and a short literature review. This phase does three things. It extends the survey to the memory operations an agent store must support and to four recent systems. It compares the substrates the same logical schema could run on: Notion databases (as used by Lore), SQLite, and PostgreSQL. Then it produces the database design: requirements, an ER diagram, the relational schema, the functional dependencies, and the normalisation argument.

The objective is a schema that Phase 3 can implement without changing it. Every table and column name in this report is the one the implementation uses.

2. Memory operations

Du and colleagues reduce memory in AI systems to six atomic operations [5]. Table 1 maps each operation to a database counterpart and to the operation our store will provide. In our design the operations are write, assert, supersede, forget, retrieve, and consolidate.

Operation [5]	Database counterpart	Planned operation in the store
Consolidation	Turning episodes into facts with keys and a validity interval	`assert`: insert a semantic fact that cites its episode
Indexing	Keyword rows, an FTS5 full-text index, an embedding per row	`write`: insert an episode and index it
Updating	Update without losing history: close the interval, insert a new row	`supersede`
Forgetting	A status flag, so the row and its foreign keys remain	`forget`
Retrieval	Query processing: predicate queries and ranked text search	`retrieve`: three ranked lists fused with RRF [6]
Compression	A materialised view: a stored query result	`consolidate`: one summary row per session

Table 1: The six memory operations of Du et al. [5] and their place in this design.

3. Related memory systems

Lore [1] is Notion's open-source memory system for agents. Memory is stored as pages in Notion databases, so a person can read and edit what the agent remembers. It ranks retrieved results with reciprocal rank fusion (RRF) [6], which combines several ranked lists by summing 1 / (k + rank). **Nemori** [2] organises memory the way cognitive science describes it. It segments conversation into episodes and uses a predict-calibrate step to decide what to keep as semantic memory. **LightMem** [3] is a lightweight memory pipeline that filters and groups input before storing it, and moves the expensive update offline into a sleep-time phase. **Harness the Memory** [4] evaluates memory substrates for memory agents and finds that no one substrate wins every task.

From these we take four decisions. Episodic and semantic memory are separate tables, as in Nemori. Retrieval fuses several ranked lists with RRF, as Lore does, with k = 60. A predicate query on current facts is kept beside the ranked text search, because no single access path wins every question [4]. Nemori's predict-calibrate loop and LightMem's sleep-time update are out of scope; consolidation stores the active facts of a session.

4. One logical schema on three substrates

The same logical schema can run on Notion databases (as in Lore), on SQLite, or on PostgreSQL. What changes is which rules the substrate itself enforces. Table 2 compares the three for the constraints this design needs.

Requirement	Lore (Notion databases)	SQLite	PostgreSQL
Readable and editable by people	Yes, as Notion pages	Through the sqlite3 shell	Through a client
Server or account needed	A Notion workspace	None: one file	A database server
Unique alias (one entity per alias)	Not declared; left to the application	PRIMARY KEY (alias_key)	PRIMARY KEY
Provenance (fact cites episode)	Not declared as a foreign key	FOREIGN KEY, enforced per connection	FOREIGN KEY
Domain and interval checks	Not declared	CHECK constraints	CHECK constraints
At most one current owner	Found by a scan afterwards	Partial unique index on active rows	Partial unique index
No two overlapping validity intervals	Not declared	Not expressible	EXCLUDE USING gist on a range
Full-text search	Notion's search endpoint	FTS5 virtual table	tsvector with a GIN index

Table 2: What each substrate can declare for the same schema.

Lore's vault stays readable, but it cannot declare these constraints. PostgreSQL can declare all of them, including the temporal exclusion constraint. SQLite can declare every one except that last one. It needs no server, so the program runs from a clone with no extra install. We choose SQLite for Phase 3. A partial unique index covers the current-owner rule, and the PostgreSQL exclusion constraint is noted as future work.

5. Requirements

5.1 Functional requirements

ID	Requirement
FR1	Record an agent, its sessions, and each turn of a session as an episode with a turn index and a role (user or assistant).
FR2	Register entities with a canonical name and aliases. Resolving an alias returns one entity. An alias cannot belong to two entities.
FR3	Assert a fact as subject, predicate, object, with a confidence, a stance, a valid-from time, and the episode it came from.
FR4	Refuse a second current object for the functional predicates `owned_by`, `is_a`, and `created_by`.
FR5	Supersede a fact: close its validity interval, insert the replacement, and link the old row to the new one.
FR6	Forget a fact or an episode without deleting it, so it is hidden from retrieval but provenance still joins.
FR7	Retrieve by keyword, by full-text search, and by vector similarity, and fuse the three rankings.
FR8	Answer what is believed now, and what was believed at a past time (an as-of query).
FR9	Consolidate a session into one stored summary of its active facts.
FR10	Refuse to delete an episode that a fact cites.

Table 3: Functional requirements.

5.2 Non-functional requirements

- Only the Python standard library and SQLite, so it needs no database server and no API key.
- Repeatable: the session uses fixed timestamps, so a second run prints the same output.
- Inspectable: every table, including stored vectors, can be read with the `sqlite3` shell.
- Every rule that can be declared in the schema is declared there, not only checked in Python, and `PRAGMA integrity_check` must pass.

6. Conceptual design

Table 4 lists the entity sets and their attributes. Primary keys are underlined. Episodic memory is `EPISODE`: one row per turn. Semantic memory is `FACT`: a triple with a confidence, a stance, and a validity interval. `ENTITY` is the registry of canonical names that facts point to.

Entity set	Attributes
AGENT	agent_id, name, created_at
SESSION	session_id, title, started_at
EPISODE	episode_id, turn_index, role, content, importance, forgotten, created_at, {keyword}
ENTITY	entity_id, name, name_key, kind, description, created_at, {alias}
FACT	fact_id, predicate, object_text, confidence, stance, valid_from, valid_until, status, statement, created_at, {keyword}
SUMMARY	summary_id, body, episode_count, created_at
EMBEDDING (weak)	embedding_id, dims, vector

Table 4: Entity sets and attributes.

Braces mark a multi-valued attribute. Table 5 lists the relationships.

Relationship	Between	Cardinality	Participation
opens	AGENT, SESSION	1 : N	Every session has an agent (total)
holds	SESSION, EPISODE	1 : N	Every episode belongs to a session (total)
yields	SESSION, SUMMARY	1 : N	Every summary belongs to a session (total)
supports	EPISODE, FACT	1 : N	A fact may cite its source episode (partial)
asserts	ENTITY, FACT	1 : N	Every fact has a subject entity (total)
refers to	ENTITY, FACT	1 : N	The object may be an entity or plain text (partial)
succeeds	FACT, FACT	1 : N	A superseded fact points to its replacement (partial)
embeds	EPISODE / FACT / SUMMARY, EMBEDDING	1 : N	Each embedding has exactly one owner (identifying)

Table 5: Relationships, cardinality, and participation.

Figure 1 is the ER diagram in Chen notation [8]. An entity is a rectangle, an attribute is an ellipse, and a relationship is a diamond. Primary keys are underlined, and cardinality is written on the lines. `alias` and `keyword` are multi-valued, so they are double ellipses. EMBEDDING depends on the episode, fact, or summary it describes, so it is a weak entity (double rectangle) and `embeds` is a double diamond. Only the main attributes are drawn; Table 4 has the full list.

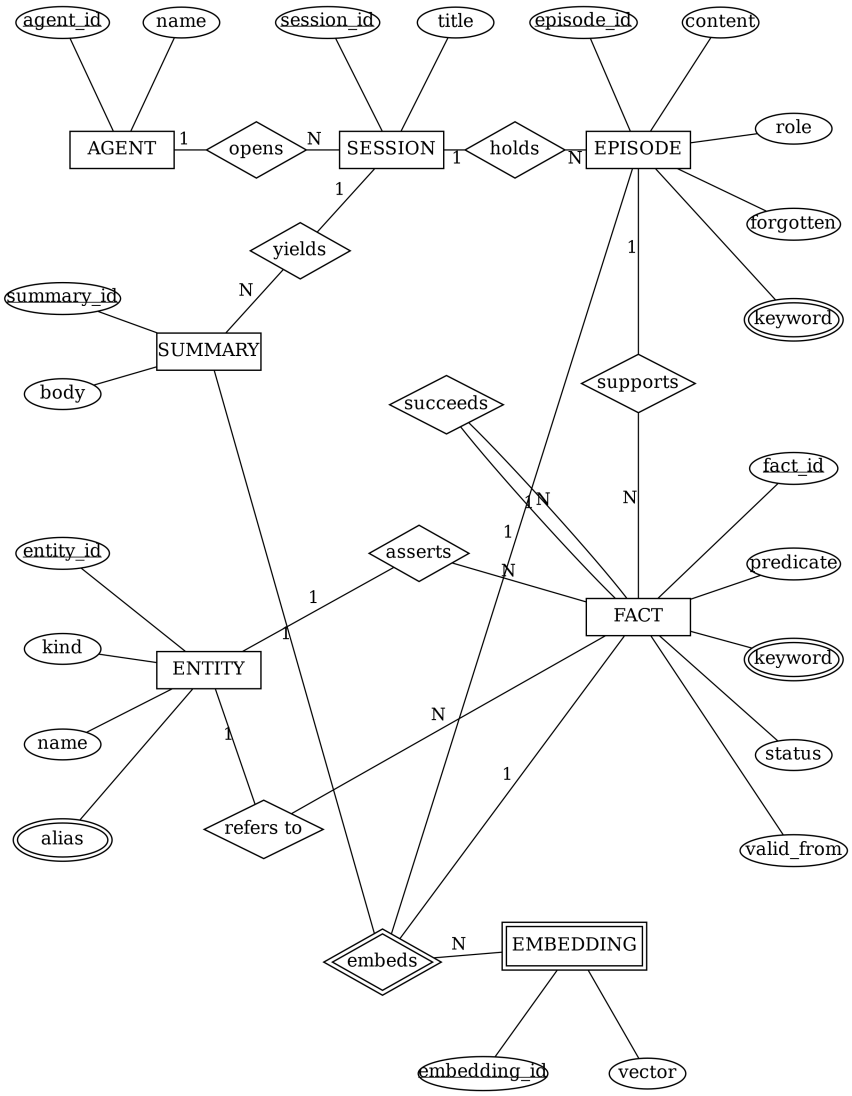


Figure 1: ER diagram of the memory store (Chen notation).

6.1 Constraints

- Domains: `role` is user or assistant; `kind` is person, team, system, project, or concept; `predicate` is is_a, uses, owned_by, created_by, or depends_on; `stance` is certain, likely, or speculative; `status` is active, superseded, or forgotten.
- Ranges: `confidence` in (0, 1]; `importance` in [0, 1]; `turn_index` >= 0; `episode_count` > 0; `dims` > 0.
- Valid time [9]: `valid_from` <= `valid_until`. An active fact has an open interval, and a superseded or forgotten fact has a closed one.
- A fact has an object: either an object entity or object text.
- Uniqueness: one turn index per session; one entity per `name_key`; one entity per alias; at most one active `owned_by`, `is_a`, or `created_by` per subject.
- A keyword belongs to exactly one episode or fact. An embedding belongs to exactly one episode, fact, or summary.

7. Logical design

Table 6 shows how each part of the ER model becomes a table, a column, or a constraint.

ER construct	Rule	Result
Strong entity sets	One table each, keyed by the entity key	agent, session, episode, entity, fact, summary
opens, holds, yields (1 : N)	Foreign key on the N side	session.agent_id, episode.session_id, summary.session_id
supports, asserts, refers to (1 : N)	Foreign keys on FACT	fact.source_episode_id, fact.subject_entity_id, fact.object_entity_id
succeeds (recursive 1 : N)	Self-referencing foreign key	fact.superseded_by references fact(fact_id)
Multi-valued alias	Own table, one row per value	entity_alias(alias_key PK, alias, entity_id FK)
Multi-valued keyword	Own table, one row per value	keyword(keyword_id, term, episode_id or fact_id), exactly one parent
Weak EMBEDDING, embeds	Table with a foreign key to each possible owner	embedding(episode_id, fact_id, summary_id), CHECK exactly one is set
Episode range of a summary	Two foreign keys to EPISODE	summary.first_episode_id, summary.last_episode_id

Table 6: ER-to-relational mapping.

The resulting relational schema [7] follows. It is printed from the catalogue of a database built from the schema file, with `PRAGMA table_info` and `PRAGMA foreign_key_list`. An arrow means a foreign key to that table.

```
agent(agent_id (PK), name, created_at)
session(session_id (PK), agent_id -> agent, title, started_at)
episode(episode_id (PK), session_id -> session, turn_index, role, content, importance,
    forgotten, created_at)
entity(entity_id (PK), name, name_key, kind, description, created_at)
entity_alias(alias_key (PK), alias, entity_id -> entity)
fact(fact_id (PK), subject_entity_id -> entity, predicate, object_text, object_entity_id ->
    entity, confidence, stance, valid_from, valid_until, status, source_episode_id -> episode,
    superseded_by -> fact, statement, created_at)
keyword(keyword_id (PK), term, episode_id -> episode, fact_id -> fact)
summary(summary_id (PK), session_id -> session, body, episode_count, first_episode_id ->
    episode, last_episode_id -> episode, created_at)
embedding(embedding_id (PK), episode_id -> episode, fact_id -> fact, summary_id -> summary,
    dims, vector)
```

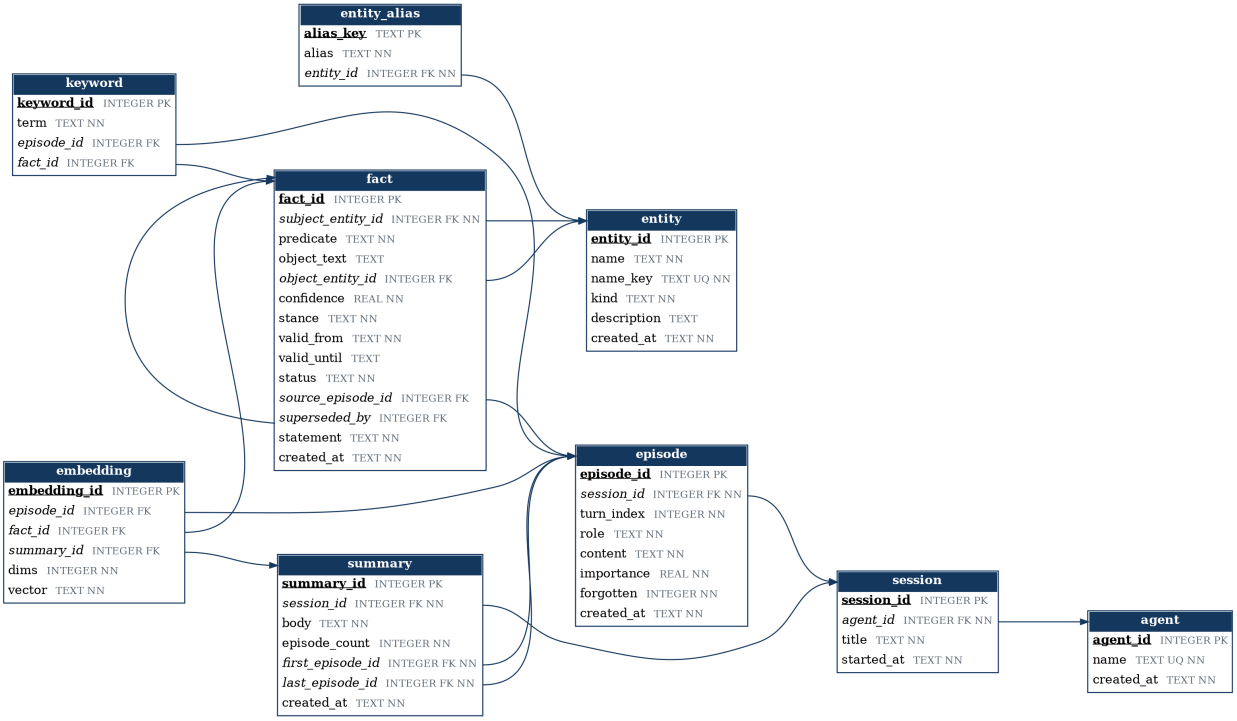


Figure 2: Relational schema. Primary keys are underlined, foreign keys are in italics, and each arrow is a foreign key. UQ is unique and NN is NOT NULL.

8. Functional dependencies and normalisation

8.1 Starting point

A memory written as notes, without a design, puts everything about one turn into one row. Table 7 shows two such rows from the worked example used throughout the project: the agent Archivist records a design session for Campus Navigator, in which ownership moves from the DBMS project team to the AI Lab.

Attribute	Row for turn 1	Row for turn 5
`agent`	Archivist	Archivist
`session_title`	Campus Navigator design	Campus Navigator design
`turn_index`	1	5
`content`	Campus Navigator is owned by the DBMS project team.	Ownership moved. The AI Lab now owns Campus Navigator.
`subject`	Campus Navigator	Campus Navigator
`subject_aliases`	Campus Navigator, navigator, the app	Campus Navigator, navigator, the app
`predicate`	owned_by	owned_by
`object`	DBMS project team	AI Lab
`valid_from`	2026-09-30T09:10:00	2026-09-30T09:40:00
`valid_until`	2026-09-30T09:40:00	NULL (open)
`keywords`	campu, navigator, own, dbm, project, team	ownership, mov, ai, lab, now, own, campu, navigator

Table 7: Two unnormalised MEMORY_NOTE rows, shown one per column. subject_aliases and keywords are repeating groups.

8.2 First normal form

The subject_aliases cell and the keywords cell each hold a list, so MEMORY_NOTE is not in 1NF. Each list becomes rows of its own table: `entity_alias(alias_key, alias, entity_id)`, one row per alias, and `keyword(keyword_id, term, episode_id, fact_id)`, one row per term. A query for "the app" is then an equality on a key, not a substring search inside a cell.

8.3 Second normal form

After 1NF the turn is identified by the composite key `(session_id, turn_index)`. The session title and the agent depend on the session alone, not on the whole key. That is a partial dependency, so it moves to `session(session_id, agent_id, title, started_at)`. The turn becomes `episode` with the surrogate key `episode_id`, and `UNIQUE (session_id, turn_index)` is kept as a candidate key.

8.4 Third normal form

Two transitive dependencies remain. The agent's name depends on the agent, which depends on the session, so it moves to `agent`. The subject's name and kind depend on the entity, not on the turn, so they move to `entity`, and the fact refers to it by `subject_entity_id`. A fact also has its own life: one turn may state no fact or several, and a fact's validity interval changes when it is superseded. So facts become their own table, `fact`, which points back to the turn through `source_episode_id`. That foreign key is the provenance link.

8.5 Boyce-Codd normal form

Table 8 lists the functional dependencies of each table. In every table, each determinant is a superkey.

Table	Functional dependencies	Candidate keys	BCNF
agent	agent_id → name, created_at; name → agent_id	{agent_id}, {name}	Yes
session	session_id → agent_id, title, started_at	{session_id}	Yes
episode	episode_id → all; (session_id, turn_index) → all	{episode_id}, {session_id, turn_index}	Yes
entity	entity_id → all; name_key → all	{entity_id}, {name_key}	Yes
entity_alias	alias_key → alias, entity_id	{alias_key}	Yes
fact	fact_id → all other attributes	{fact_id}	Yes, except the derived statement (8.6)
keyword	keyword_id → term, episode_id, fact_id	{keyword_id}	Yes
summary	summary_id → all	{summary_id}	Yes
embedding	embedding_id → all	{embedding_id}	Yes

Table 8: Functional dependencies and BCNF check.

8.6 Derived columns kept on purpose

- `entity.name_key` and `entity_alias.alias_key` are the case-folded name and alias, so "AI Lab" and "ai lab" cannot become two rows. They are derived, and they are what the uniqueness is declared on.
- `fact.statement` is the sentence made from the subject name, the predicate, and the object. It is determined by the triple, which is not a key, so strictly it breaks BCNF. It is kept because the full-text index reads its text from a column of the base table.

- `summary.body` and `episode_count` are a snapshot taken at consolidation. They are a stored query result (a materialised view), not a second copy that has to stay in step.
- `embedding.vector` is computed from the row's text and stored as one value. The database never queries a single element of it, so it is treated as atomic.

9. Physical design plan

Table 9 lists the indexes and other physical structures the schema declares. Besides these, the views `belief_history` (every fact with subject and object names) and `current_belief` (active facts only) give the current-belief and as-of queries a stable shape.

Structure	On	Purpose
`ux_one_active_functional_fact`	fact (subject_entity_id, predicate)	Partial unique index: at most one active owned_by, is_a, or created_by per subject (FR4).
`ix_fact_source`	fact (source_episode_id)	Provenance joins from a fact to its episode.
`ix_fact_validity`	fact (valid_from, valid_until)	As-of queries on the validity interval (FR8).
`ix_fact_object`	fact (object_entity_id)	Facts that refer to an entity as the object.
`ix_keyword_term`	keyword (term)	Keyword lookup (FR7).
`episode_fts`	episode (content)	FTS5 full-text index, external content, porter tokenizer; kept in step by triggers.
`fact_fts`	fact (statement)	FTS5 full-text index over fact statements; kept in step by triggers.

Table 9: Indexes and full-text structures, read from the schema catalogue.

The rule behind FR4 is declared as a partial unique index:

```
-- At most one *current* object for predicates that are functions of the
-- subject. This is the constraint a conflict scan otherwise has to rediscover.
CREATE UNIQUE INDEX ux_one_active_functional_fact
  ON fact (subject_entity_id, predicate)
  WHERE status = 'active'
    AND predicate IN ('owned_by', 'is_a', 'created_by');
```

A vector is planned as a signed feature hash of the row's tokens into 512 buckets [10], stored as a JSON array. That needs no model and no network call. A real embedding model can replace it later without changing the schema. The as-of query planned for FR8 reads the history view:

```
SELECT subject, object, status, valid_from, valid_until
FROM belief_history
WHERE subject = 'Campus Navigator'
  AND predicate = 'owned_by'
  AND valid_from <= '2026-09-30T09:30:00'
  AND (valid_until IS NULL OR valid_until > '2026-09-30T09:30:00');
```

The other planned queries list current beliefs, list aliases one row per alias, join each active fact to the episode that supports it, list the foreign keys on `fact`, and run `PRAGMA integrity_check`.

10. Recommendation and plan for Phase 3

The recommendation is a local relational store with episodic rows, semantic triples with a validity interval, a normalised entity and alias table, provenance back to the episode that supported a claim, and indexes for keyword and vector retrieval. Phase 3 implements this schema in SQLite as `schema.sql`, with a Python

module for write, assert, supersede, forget, retrieve, and consolidate, and a scripted agent session that exercises every requirement in Table 3 and prints checks.

References

[1] Notion (makenotion). *lore: persistent, shared AI memory backed by Notion*. GitHub, MIT licence, commit 95c3558. https://github.com/makenotion/lore

[2] W. Ma, J. Nan, W. Wu, and Y. Chen. *Nemori: Self-Organizing Agent Memory Inspired by Cognitive Science*. arXiv:2508.03341, 2025.

[3] J. Fang, X. Deng, H. Chen, N. Zhang, et al. *LightMem: Lightweight and Efficient Memory-Augmented Generation*. ICLR 2026. arXiv:2510.18866.

[4] W.-C. Huang, W. Zhang, Y. Wu, Y. Chen, et al. *Harness the Memory: A Holistic Evaluation of Memory Substrates in Memory Agents*. arXiv:2608.15008, 2026.

[5] Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong, and J. Z. Pan. *Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions*. arXiv:2505.00675, 2025.

[6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR 2009.

[7] E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM 13(6):377-387, 1970.

[8] P. P.-S. Chen. *The Entity-Relationship Model: Toward a Unified View of Data*. ACM Transactions on Database Systems 1(1):9-36, 1976.

[9] K. Kulkarni and J.-E. Michels. *Temporal Features in SQL:2011*. ACM SIGMOD Record 41(3):34-43, 2012.

[10] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg. *Feature Hashing for Large Scale Multitask Learning*. ICML 2009.